#D Fail if none of the permissions for the user or groups allow this action.

You may be wondering why you would split out looking up the user's groups during authentication to then just use them in a second query against the `permissions` table during access control. It would be more efficient to instead perform a single query that automatically checked the groups for a user using a `JOIN` or sub-query against the group membership table, like the following:

```
SELECT perms FROM permissions
 WHERE space_id = ?
   AND (user_or_group_id = ?     #A
    OR user_or_group_id IN     #B
    (SELECT DISTINCT group_id    #B
       FROM group_members    #B
      WHERE user_id = ?))    #B
```

#A Check for permissions for this user directly.
#B Check for permissions for any groups the user is a member of.

Although this query is more efficient, it is unlikely that the extra query of the original design will become a significant performance bottleneck. But combining the queries into one has a significant drawback in that it violates the layering of authentication and access control. As far as possible, you should ensure that all user attributes required for access control decisions are collected during the authentication step, and then decide if the request is authorized using these attributes. As a concrete example of how violating this layering can cause problems, consider what would happen if